Go Backend

net/http, mux, Gin

Server

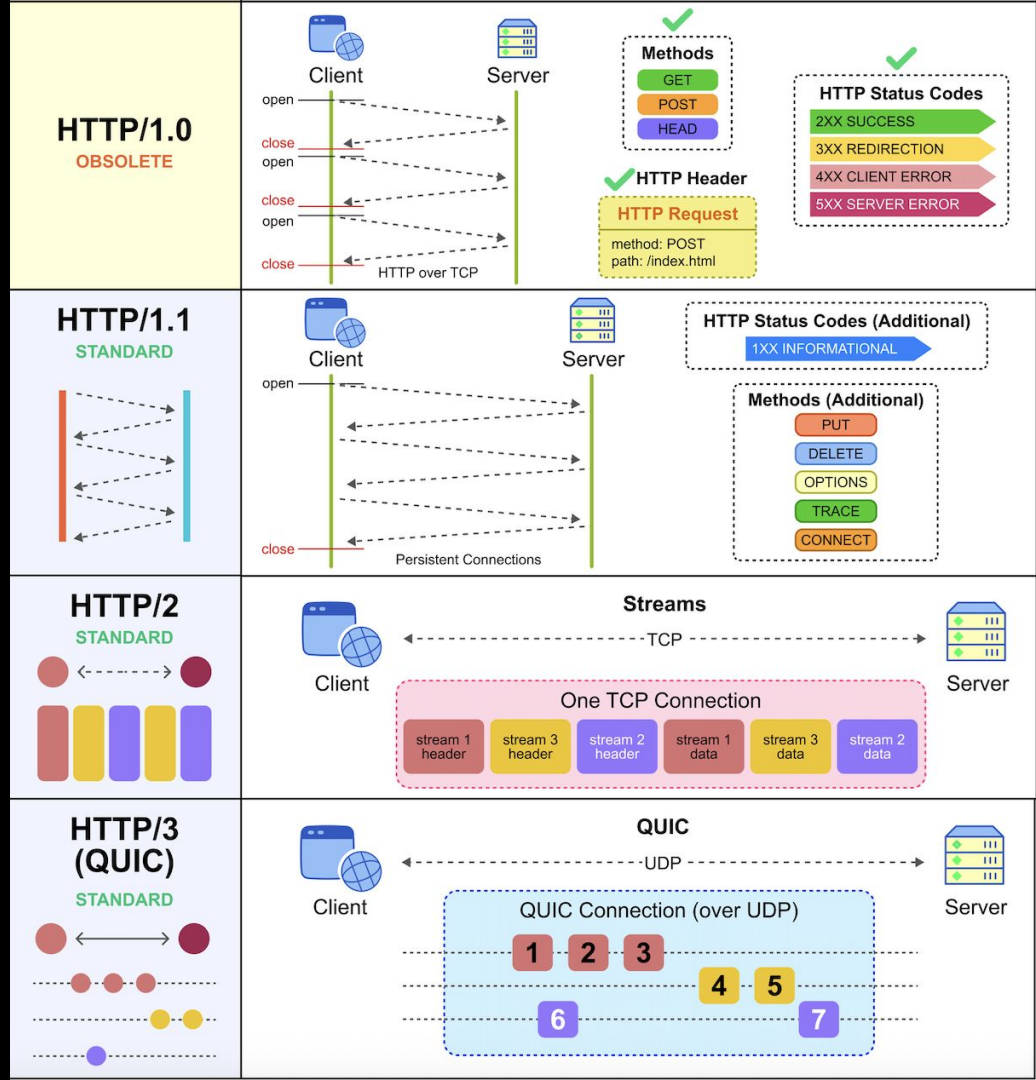
- Server
- Port
- Handler
- Router
- JSON response

http

1990s : HTTP/1.1 (by-default, common)

2015 : HTTP/2

2019: HTTP/3



net/http server

```go
func main(){
    const port = 8090
    fmt.Println("Server with mux booting on port", port)

    mux:= http.NewServeMux()

    // Request and Response
    mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        if r.Method != http.MethodGet{
            w.WriteHeader(http.StatusMethodNotAllowed)
            // w.Write([]byte("Only GET requests are allowed"))
            fmt.Fprintf(w, "Only GET requests are allowed")
            return
        }
        name := r.URL.Query().Get("name")
        if name == "" {
            name = "Stranger"
        }
        // w.Write([]byte("Hello, Web!"))
        fmt.Fprintf(w, "Hello, %s!", name)
    })

    mux.HandleFunc("/api/user", func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json")
        user := User{ID:1, Name:"Polars"}
        json.NewEncoder(w).Encode(user)
    })

    http.ListenAndServe(":8090", mux)
}
```

net/http

1. Handler
   a. ***http.ResponseWriter***
   b. ****http.Request***
2. Routing: **http.HandleFunc("/path", handlerFunc)**
3. Modern backends don't speak plain text; they speak JSON
4. Mux
   a. *DefaultServeMux*
   b. *Multiplexer*

Key concepts:

- **http.ResponseWriter** → response
- ***http.Request** → request data
- Each request handled in separate goroutine

Two most important objects in Go web development

r *http.Request (The Inbox):

- This is a pointer to a struct holding everything the user sent you.
- The URL they visited, the type of method they used (GET, POST), any query parameters (?name=John), and the body of their request.

w http.ResponseWriter (The Outbox):

- This is the interface you use to construct the reply.
- You use it to set status codes (like w.WriteHeader(404)), set headers, and write the actual data (HTML, JSON, or Text) back to the user's browser.
- w.Write(...): This is the lowest-level, rawest way to send data. It strictly requires a slice of bytes ([]byte). It's fast but annoying for mixing strings and variables together.

ResponseWriter: Important Interview Pointers

- http.ResponseWriter implements **io.Writer**
- ResponseWriter = output stream to client
- ResponseWriter is:
  - *NOT a buffer you read from*
  - *NOT a struct you manipulate directly*

Note: http.ResponseWriter is an abstraction over the HTTP response that behaves like an io.Writer, so anything that can write bytes can directly send data to the client.

```go
type ResponseWriter interface {
    Header() Header
    Write([]byte) (int, error)
    WriteHeader(statusCode int)
}
```

Practice Questions

1. Write a simple net/http server that returns "Hello, Web!" on /.
2. What happens if you don’t call WriteHeader()?

Interview Questions

1. Struct vs Interface
2. Why do we pass ":8080" instead of 8080?
3. How do you send a JSON response in net/http?
4. Does Go support HTTP/2 in net/http?

Gin

Why Gin? Or Pain Points of net/http?

Verbose routing

Manual JSON parsing

No built-in middleware chain

Boilerplate everywhere